icam
L'art et la manière de faire monde

# Soutenance Finale
# *Jumeaux Numériques*


- *Clément MADORRE*
- *Dildar AYDIN*
- *Samuel VIDAL*
- *Guillaume DE CIBEINS*
- *Inès TISSOT*

**Groupe n°14**

Promo : **A126**


Date : **27/06/2025**

# SOMMAIRE

Introduction

1) **Jumeau numérique**
   a)Modélisation

2) **Jumeau physique**
   a)Conception
   b)Développement durable
   c)Electronique/montage
   d)Programmation
   e)Vidéo demonstration

1) **Partie IA**
   a)Création d'une base de données
   b)Prédictions des températures
   c)Importation Fichier CSV IA

# Introduction

**Objectifs :**

- Concevoir un échangeur thermique à ailettes performant dont le jumeau numérique permet de prédire la température et d'optimiser la régulation.
- Création d'un système fonctionnant avec une régulation de température classique.
- Attendus de cette période :
  - Ajout des ailettes
  - Rendu final du système
  - **Intégration de l'IA**

# Présentation de l'équipe projet

**Guillaume DE CIBEINS**
Responsable montage

**Dildar AYDIN**
Technicien électronique et aide IA

**Inès TISSOT**
Responsable interface et communication

**Samuel VIDAL**
Responsable mécanique et modélisation

**Clément MADORRE**
Chef de projet (responsable électronique, électrique, programmeur, IA)

# Organisation

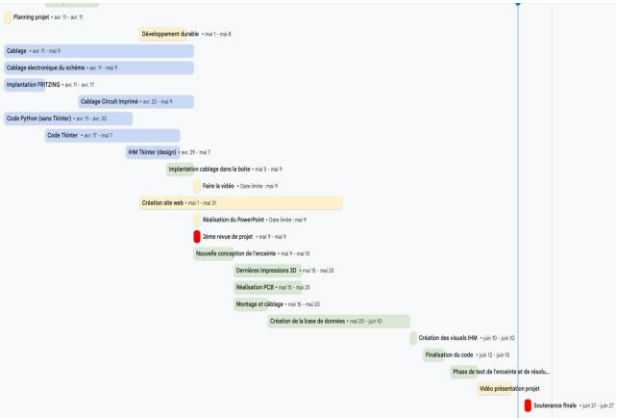
- S1 : Avancées numériques
- S2 : Avancée techniques et mécaniques

Partie numérique — Partie mécanique

| | 2024 | | | | | | | | | | | | | | | | 2025 | | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Sept | | | | Oct | | | | | Nov | | | | Déc | | | | Jan | | | | | Fév | | | | Mars | | | | Avril | | | | Mai | | | | | Juin | | | | |
| Sem | 36 | 37 | 38 | 39 | 40 | 41 | 42 | 43 | 44 | 45 | 46 | 47 | 48 | 49 | 50 | 51 | 52 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 |
| Lu | 2 | 9 | 16 | 23 | 30 | 7 | 14 | 21 | 28 | 4 | 11 | 18 | 25 | 2 | 9 | 16 | 23 | 30 | 6 | 13 | 20 | 27 | 3 | 10 | 17 | 24 | 3 | 10 | 17 | 24 | 31 | 7 | 14 | 21 | 28 | 5 | 12 | 19 | 26 | 2 | 9 | 16 | 23 | 30 |
| Ma | 3 | 10 | 17 | 24 | 1 | 8 | 15 | 22 | 29 | 5 | 12 | 19 | 26 | 3 | 10 | 17 | 24 | 31 | 7 | 14 | 21 | 28 | 4 | 11 | 18 | 25 | 4 | 11 | 18 | 25 | 1 | 8 | 15 | 22 | 29 | 6 | 13 | 20 | 27 | 3 | 10 | 17 | 24 | 1 |
| Me | 4 | 11 | 18 | 25 | 2 | 9 | 16 | 23 | 30 | 6 | 13 | 20 | 27 | 4 | 11 | 18 | 25 | 1 | 8 | 15 | 22 | 29 | 5 | 12 | 19 | 26 | 5 | 12 | 19 | 26 | 2 | 9 | 16 | 23 | 30 | 7 | 14 | 21 | 28 | 4 | 11 | 18 | 25 | 2 |
| Je | 5 | 12 | 19 | 26 | 3 | 10 | 17 | 24 | 31 | 7 | 14 | 21 | 28 | 5 | 12 | 19 | 26 | 2 | 9 | 16 | 23 | 30 | 6 | 13 | 20 | 27 | 6 | 13 | 20 | 27 | 3 | 10 | 17 | 24 | 1 | 8 | 15 | 22 | 29 | 5 | 12 | 19 | 26 | 3 |
| Ve | 6 | 13 | 20 | 27 | 4 | 11 | 18 | 25 | 1 | 8 | 15 | 22 | 29 | 6 | 13 | 20 | 27 | 3 | 10 | 17 | 24 | 31 | 7 | 14 | 21 | 28 | 7 | 14 | 21 | 28 | 4 | 11 | 18 | 25 | 2 | 9 | 16 | 23 | 30 | 6 | 13 | 20 | 27 | 4 |
| | A3.6/A4.7 | | | | A4.7 | | | | | | | | | | | | | | | | | | | | | | | | | | A4.8 | | | | | | | | | | | | | |

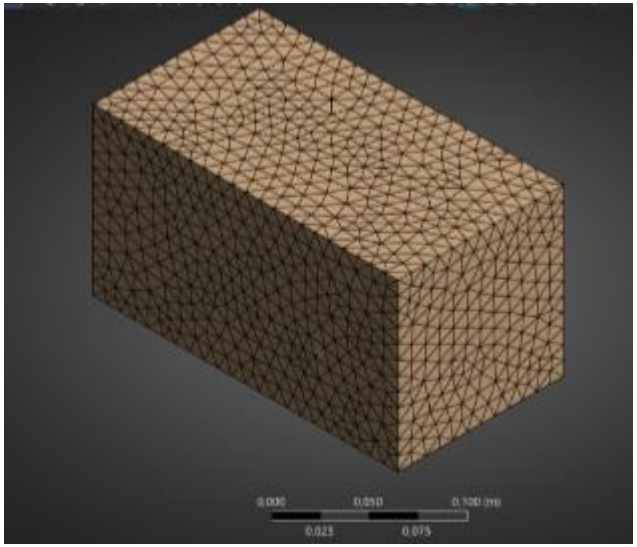
Ansys — Conception — SolidWorks — Programmation — Cablâge — Enceinte et tests

- Travail d'équipe primordial :
  - Communication (drive partagé, groupes…)
  - Gestion de projet via un planning GANTT
  - Répartition des tâches et mise en commun

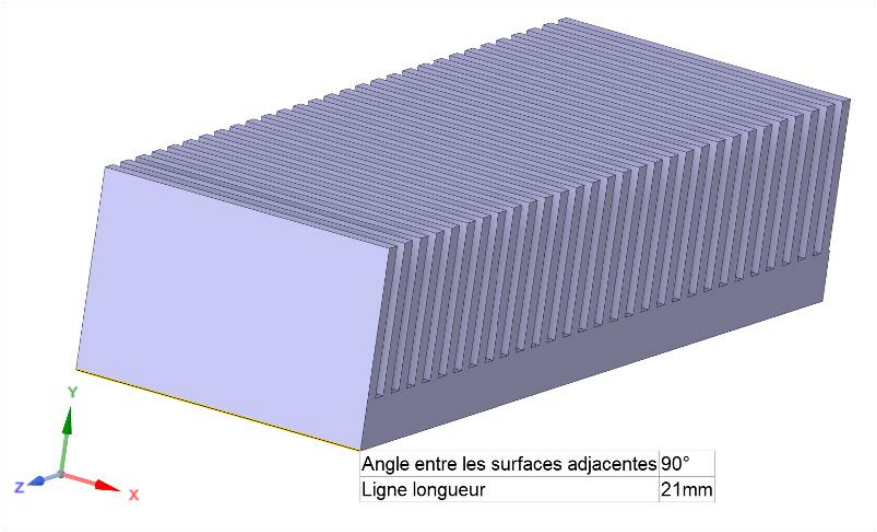


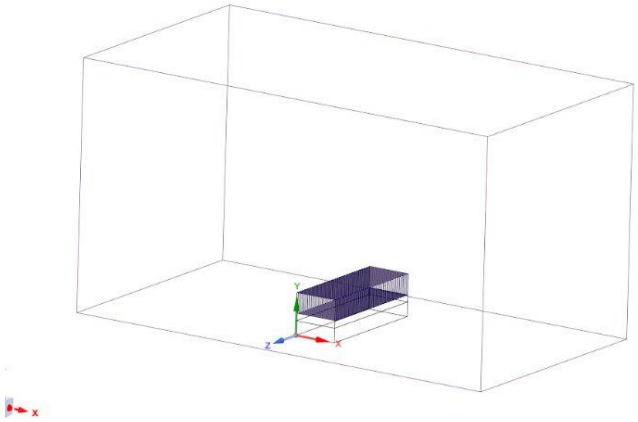
# 1) Jumeau Numérique

# Jumeau Numérique



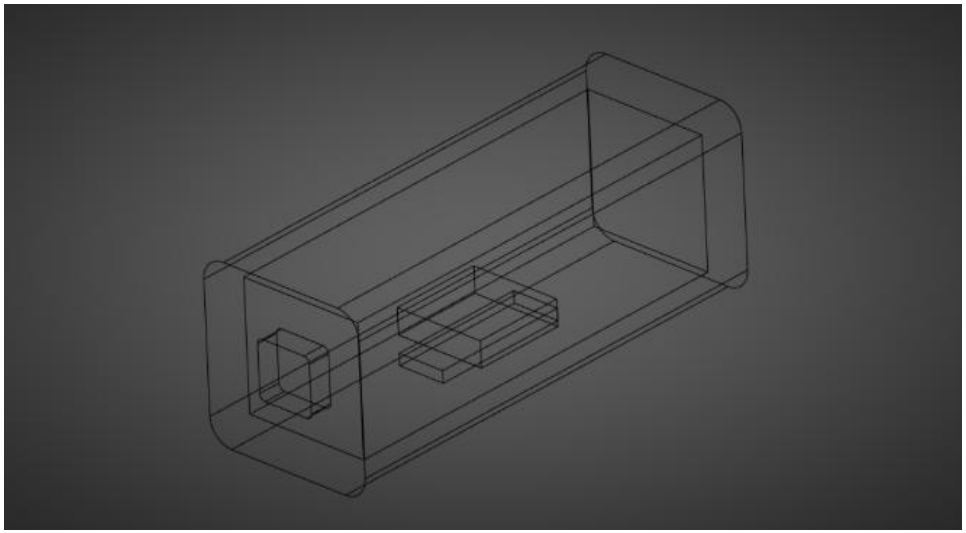
S1



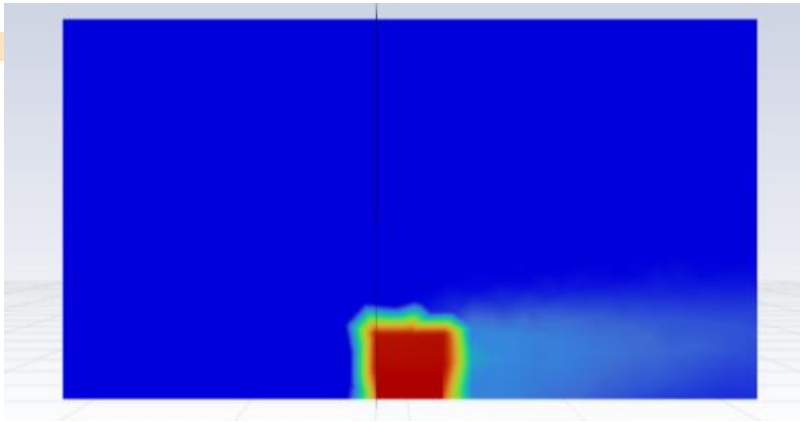



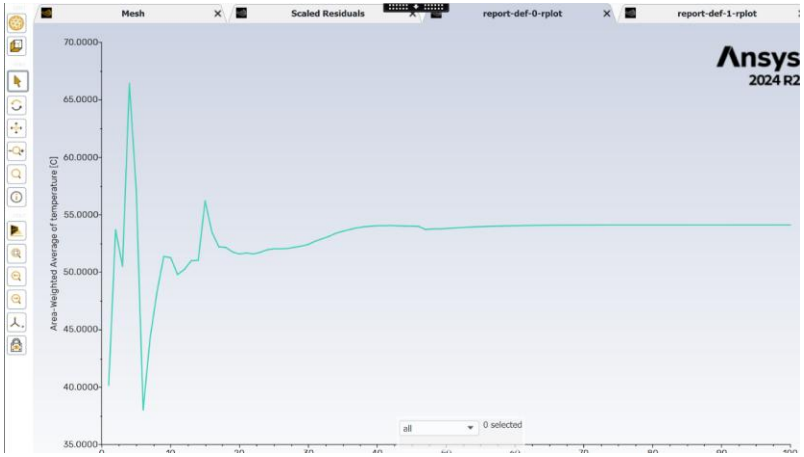
Première idée : installation d'une ailette

# Simulation du modèle



1er modèle





# 2) Jumeau Physique

# Développement durable

- Consommation d’énergie - Système d'hystérisis à 2°C : rend le système plus stable en empêchant des changements trop fréquents, surtout quand la température fluctue légèrement autour de la consigne.
- Division / 2 de la taille du jumeau physique
- Economie de matériaux
- Utilisation de PLA pour l’impression
- Utilisation de chute pour la plaque
- Economie d’essais physiques infructueux
- Beaucoup d'électronique

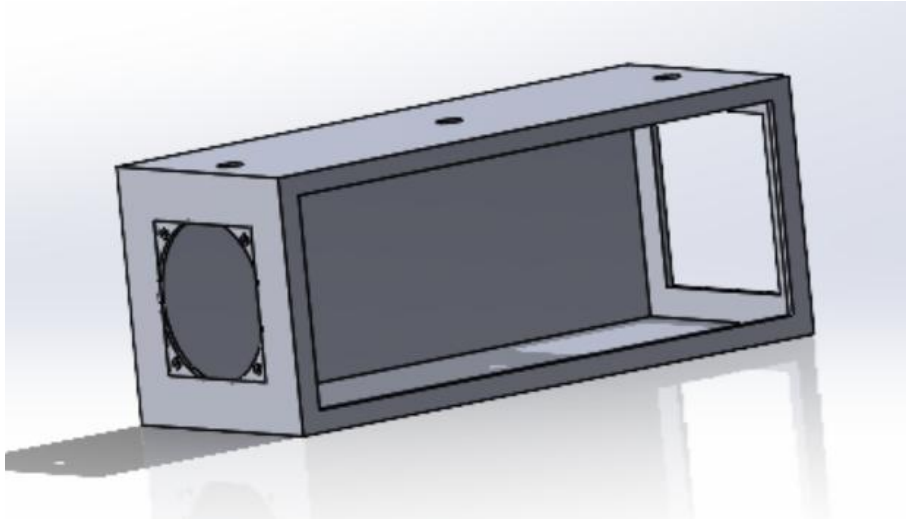
```python
# Gestion de la régulation avec hystérésis
if self.regulation:
    consigne = self.consigne.get()
    # Seuil d'activation: température > consigne + 2°C
    # Seuil de désactivation: température < consigne - 2°C
    if t2 > consigne + 2:
        # Activer le ventilateur si température > consigne + 2°C
        if not self.ventilateur_etat:
            self.ventilateur_etat = True
            self.status_ventil_label.config(text="Ventilateur : ON (régulation)", foreground='green')
    elif t2 < consigne - 2:
        # Désactiver le ventilateur si température < consigne - 2°C
        if self.ventilateur_etat:
            self.ventilateur_etat = False
            self.status_ventil_label.config(text="Ventilateur : OFF (régulation)", foreground='red')
```

# Conception - amélioration du système

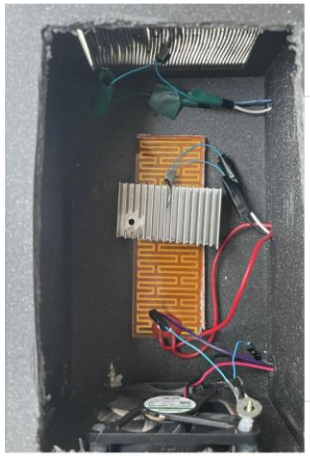
- Conception Solidworks du boîtier

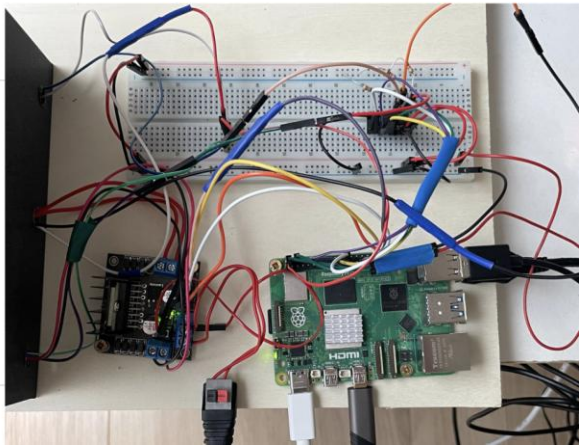
  → éviter les erreurs de conception et dimensionnement

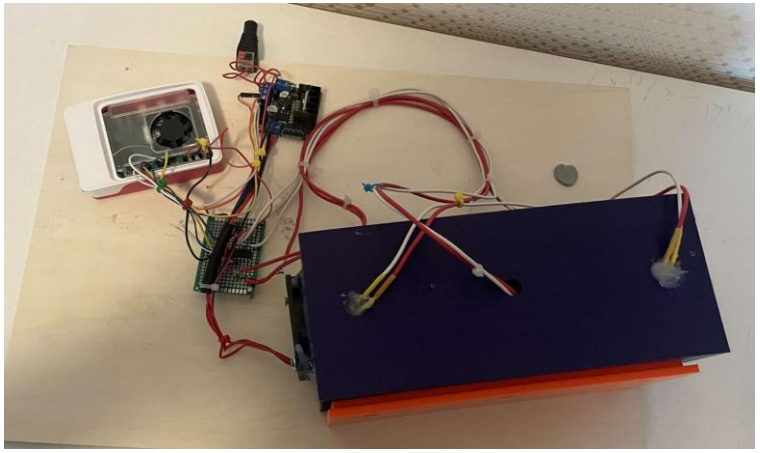
- Impression 3D FABLAB
- Ajustements

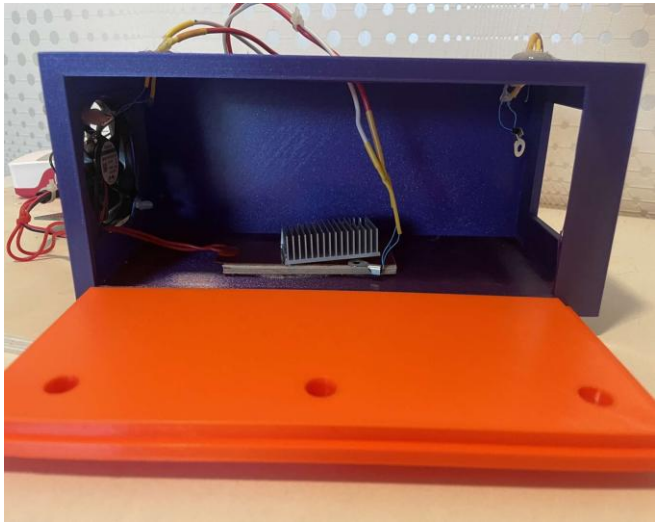


# Amélioration du système



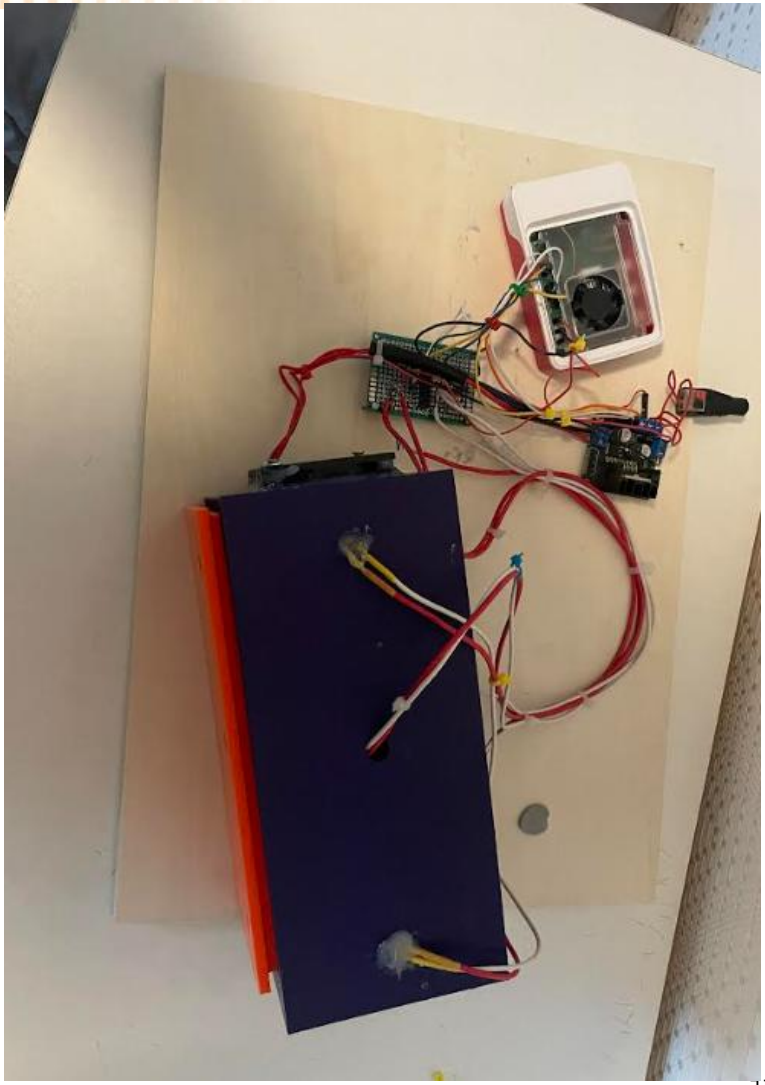




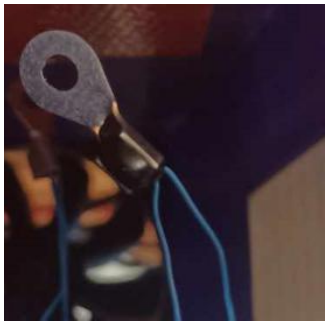


# Conception electronique

- Affichage des 3 températures
  - Entrée
  - Résistance
  - Sortie
- Contrôle manuel du ventilateur et de la résistance thermique
- Régulation automatique à la température de consigne



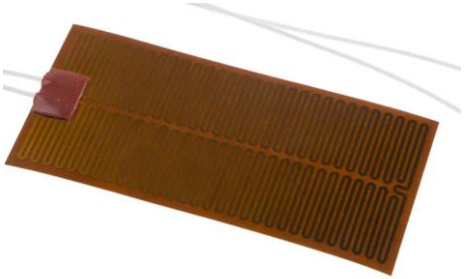
# Composants



3 Sondes NTC

Résistance 10K

Ventilateur 60x60 mm

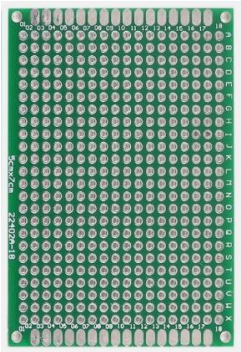


Résistance chauffante carré

Raspberry Pi 5

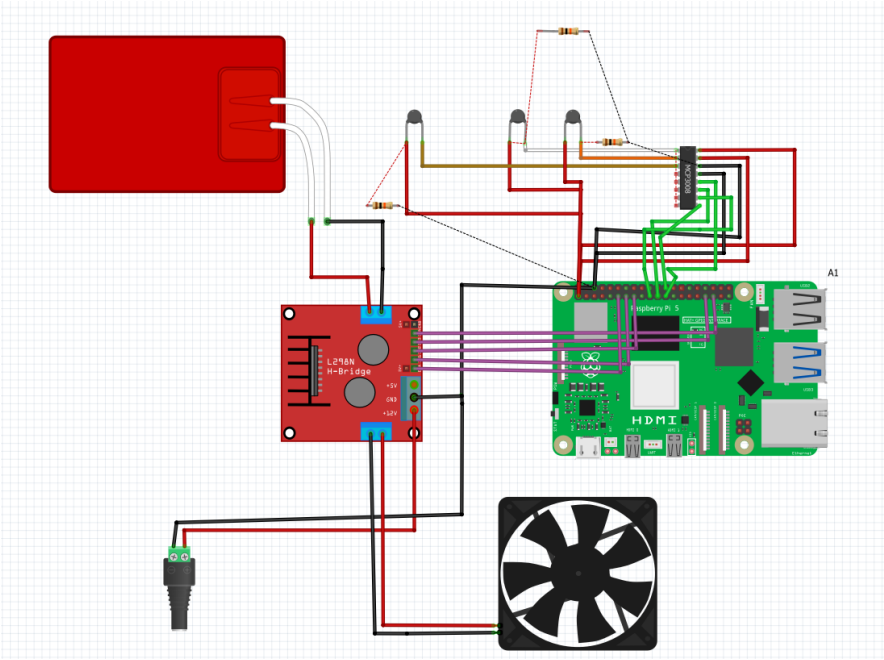
ADC MCP3008

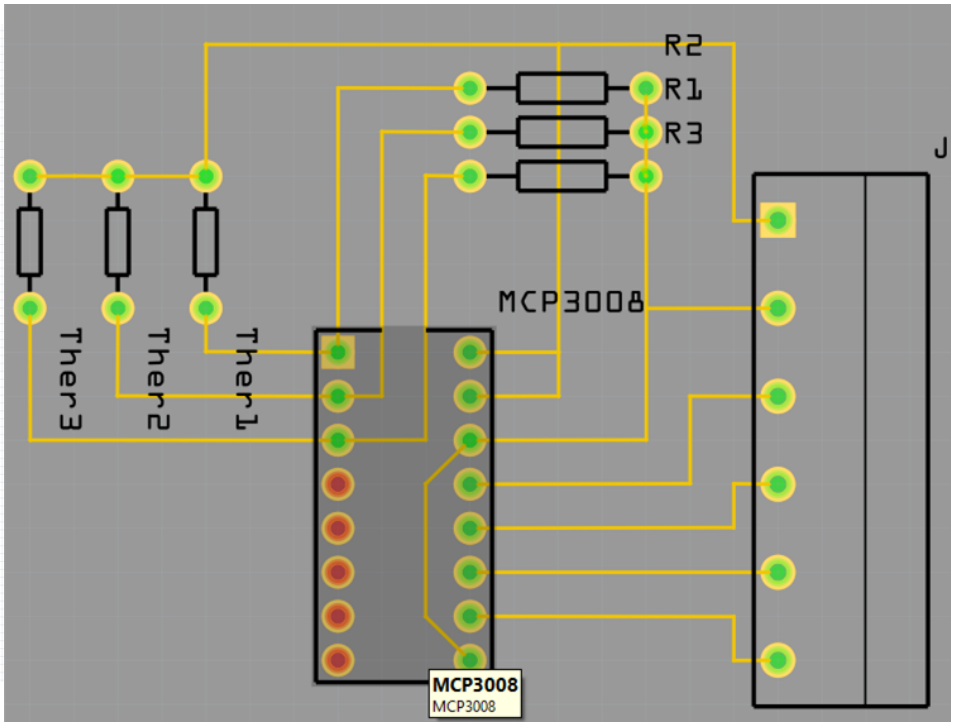
Pont en H



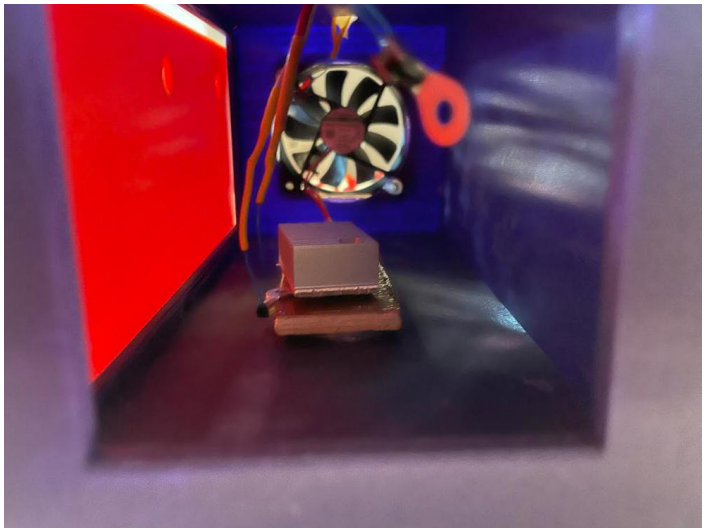
Carte PCB

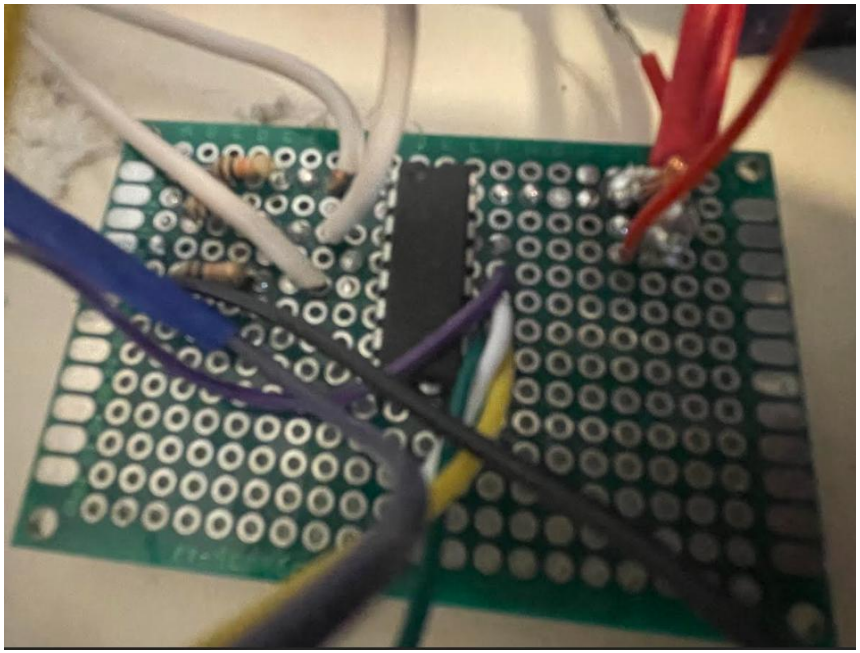
# Schéma électrique Platine/PCB

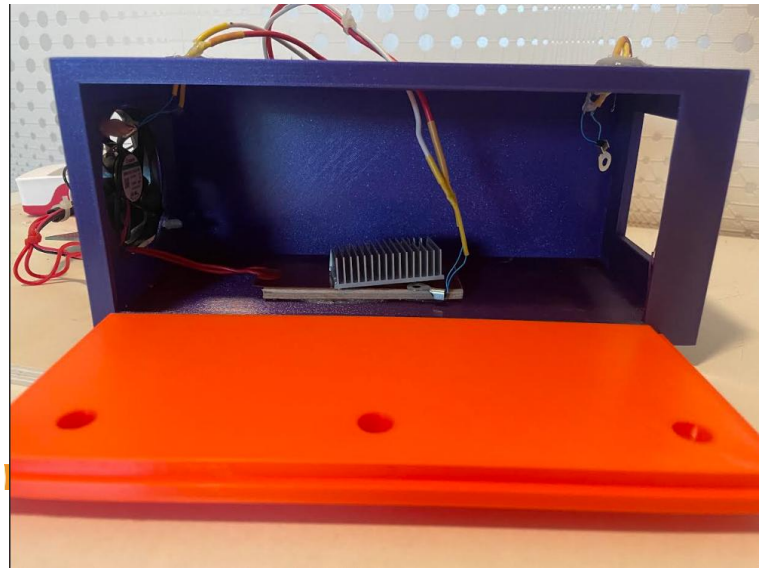


# Montage reel

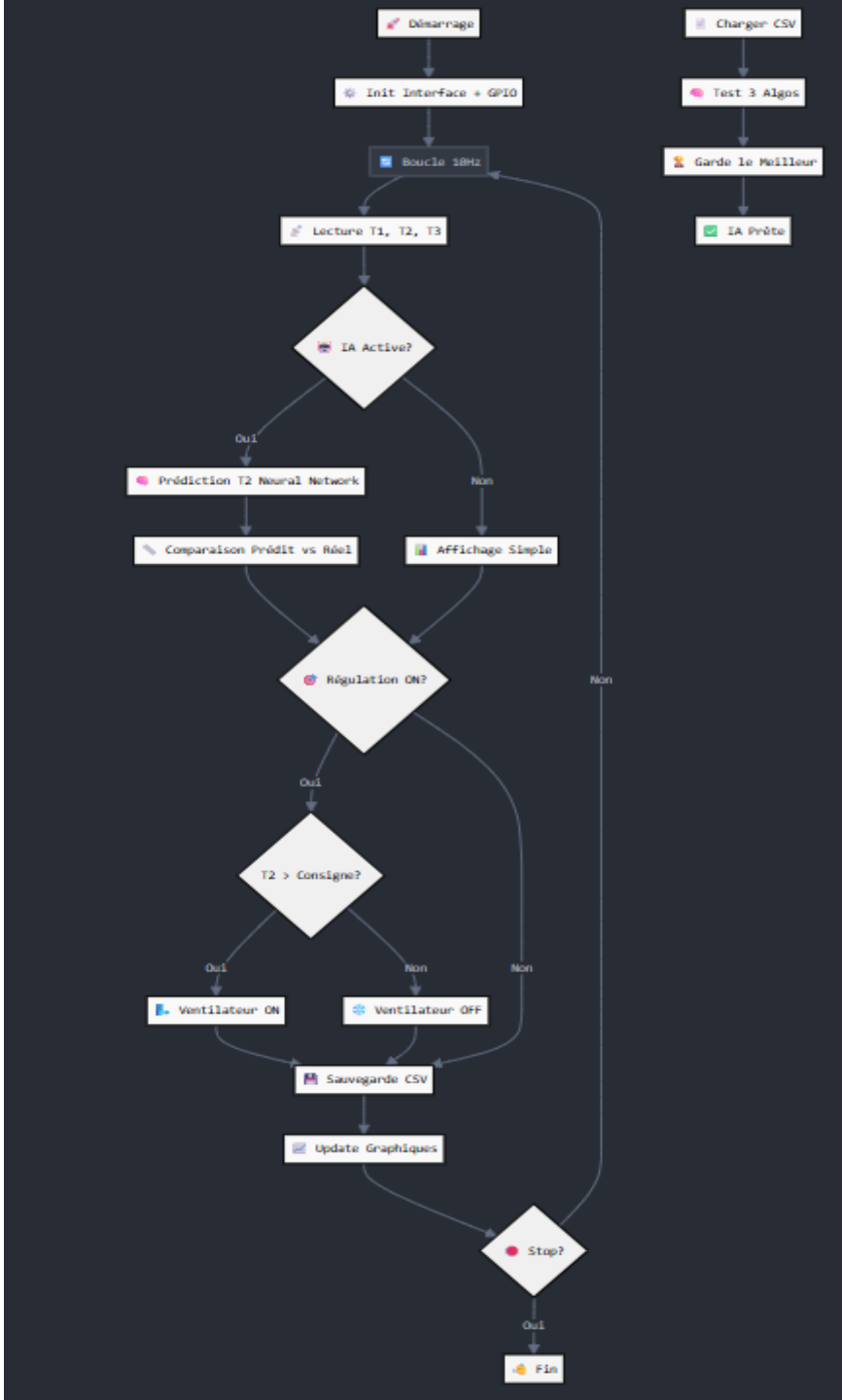




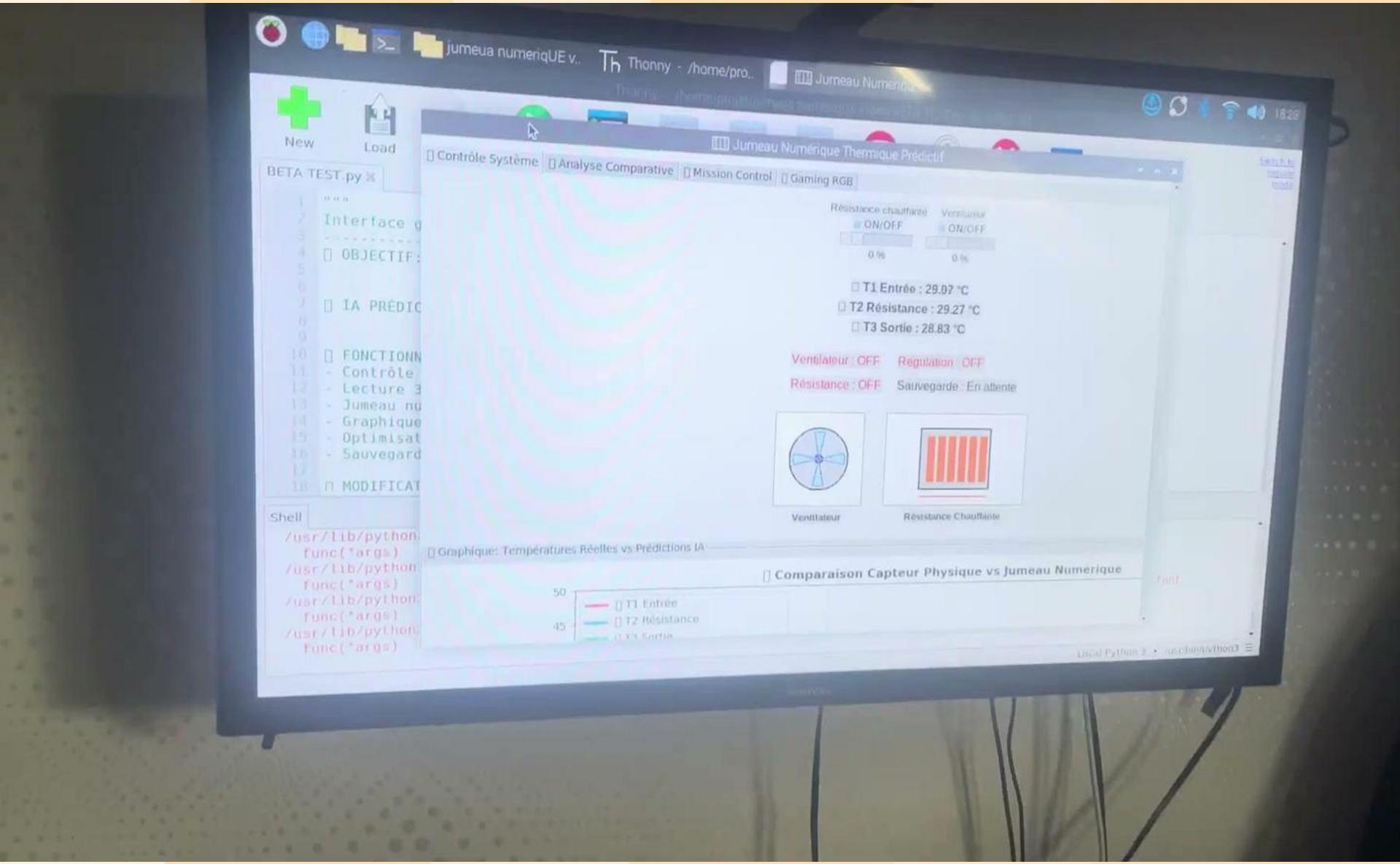


# Programmation

```
🚀 === DÉMARRAGE JUMEAU NUMÉRIQUE THERMIQUE OPTIMISÉ ===
🎯 Objectif: Prédiction T2 via apprentissage automatique
📊 Données: Capteurs physiques vs Prédictions IA
🤖 Algorithmes: RandomForest, GradientBoosting, Neural Network
🔧 OPTIMISATIONS:
   📝 Enregistrement: toutes les 5s (compromis optimal)
   💾 Sauvegarde sécurité: toutes les 5 minutes
   🌬️ Ventilateur auto: 66-100% (plage optimisée)
============================================================
✅ Image ventilateur chargée: c:\Users\cleme\OneDrive\Documents\ventillo.png
✅ Images radiateur chargées avec succès
```



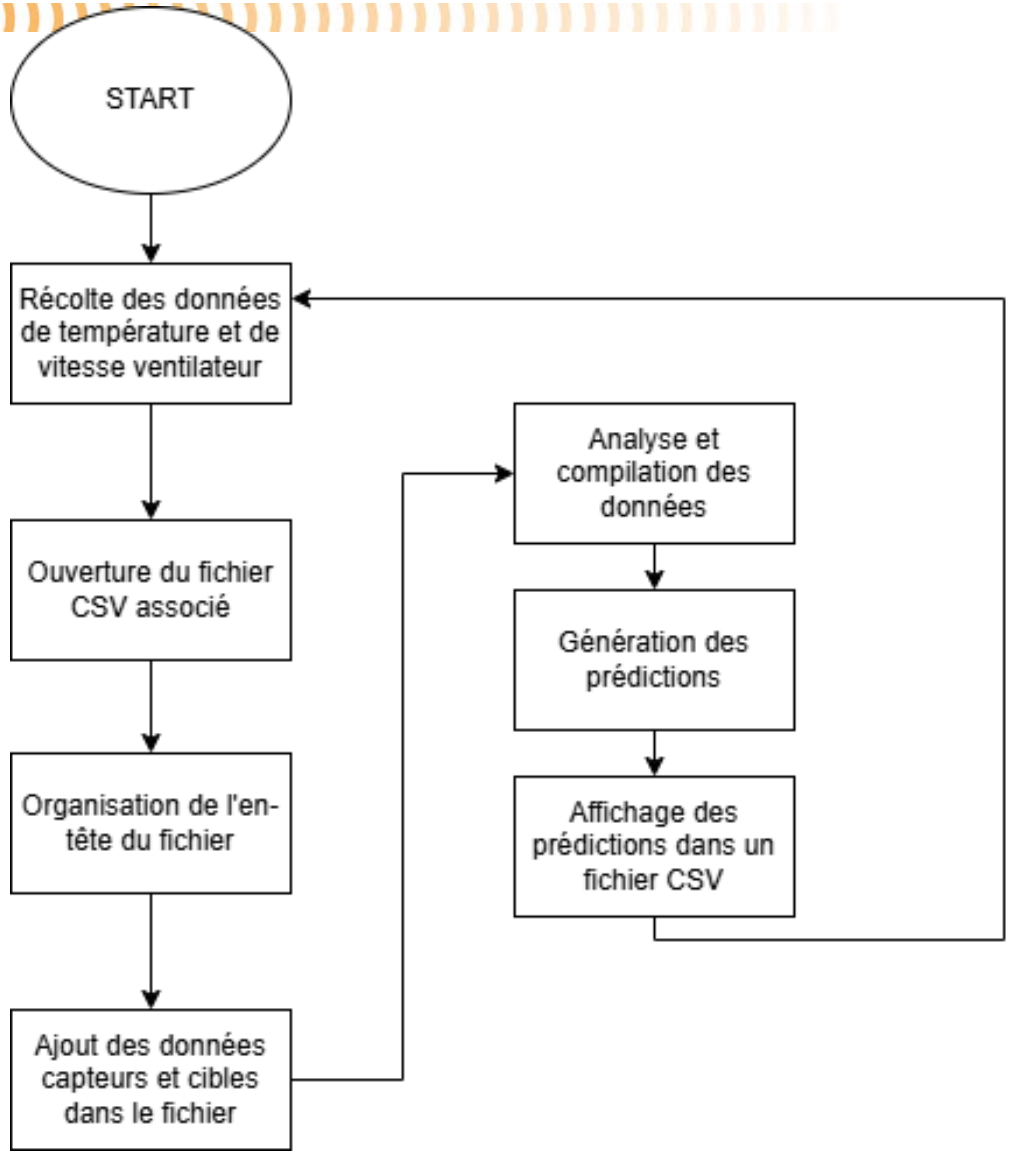
# Vidéo de démonstration



# 3) Partie IA

# Création d'une base de données

1. **Definir**



# Création d'une base de données

## 2. Le Chargement des données

```
import pandas as pd
```

Avant tout, il faut préparer les données stockées dans un fichier csv. On définit les variables d'entrée du modèle et la variable de sortie.

```
timestamp = datetime.datetime.now().strftime("%Y-%m-%d_%H-%M-%S")
filename = f"donnees_thermiques_backup_{timestamp}.csv"

with open(filename, "w", newline='') as f:
    writer = csv.writer(f, delimiter=';')
    writer.writerow([
        "Temps", "T1", "T2", "T3", "Consigne",
        "Puissance Chauffe", "Puissance Vent",
        "Ventilateur ON", "Chauffage ON", "Regulation Active"
    ])
```

# IA de prédiction de température

**3. Importation**: 3 modèles de machine learning - deux à base d'arbres (Random Forest et Gradient Boosting), et un réseau de neurones. Utilisation des outils pour diviser les données, évaluer les performances de l'IA et les normaliser. Tout ça me permet d'entraîner différents modèles, de comparer leurs performances et de choisir automatiquement le plus performant pour la prédiction finale.

```
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegresso
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.preprocessing import StandardScaler
```

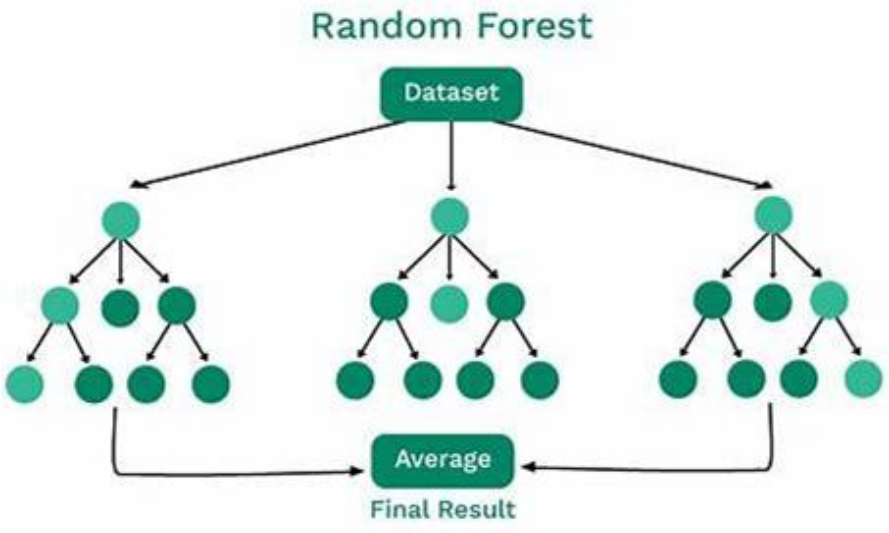
```
# ⏳ COMPÉTITION DES ALGORITHMES
modeles_candidats = {
    'RandomForest': RandomForestRegressor(
        n_estimators=100, max_depth=15, random_state=42
    ),
    'GradientBoosting': GradientBoostingRegressor(
        n_estimators=100, max_depth=8, learning_rate=0.1, random_state=42
    ),
    'NeuralNetwork': MLPRegressor(
        hidden_layer_sizes=(100, 50, 25), max_iter=1000,
        random_state=42, early_stopping=True
    )
}
```

# IA de prédiction de température

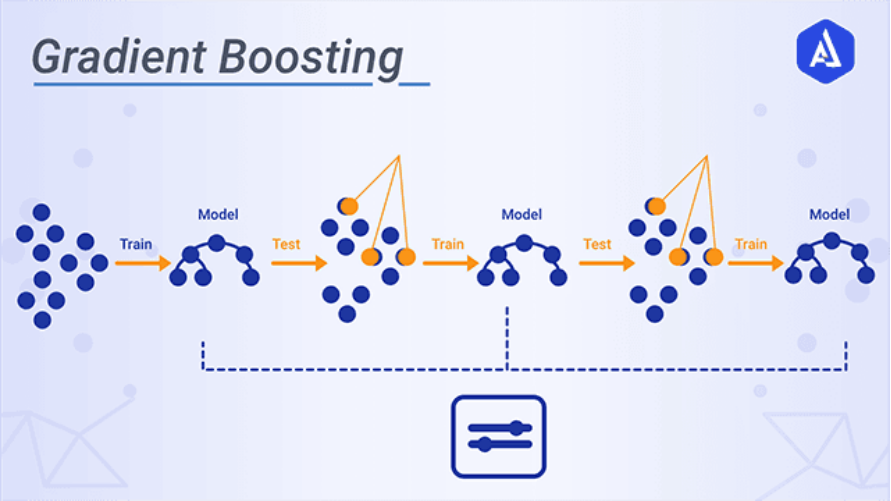
Le Random Forest, c'est un algorithme supervisé qu'on utilise pour faire de la régression.
Il crée plein d'arbres de décision, chacun entraîné sur des données tirées au hasard.

- À chaque étape, il choisit aussi des variables au hasard pour éviter que tous les arbres se ressemblent.
- Pour prédire, chaque arbre donne une réponse, et on fait la moyenne.

Ça rend le modèle plus stable, plus précis, même si les données sont un peu bruyantes.



# IA de prédiction de température



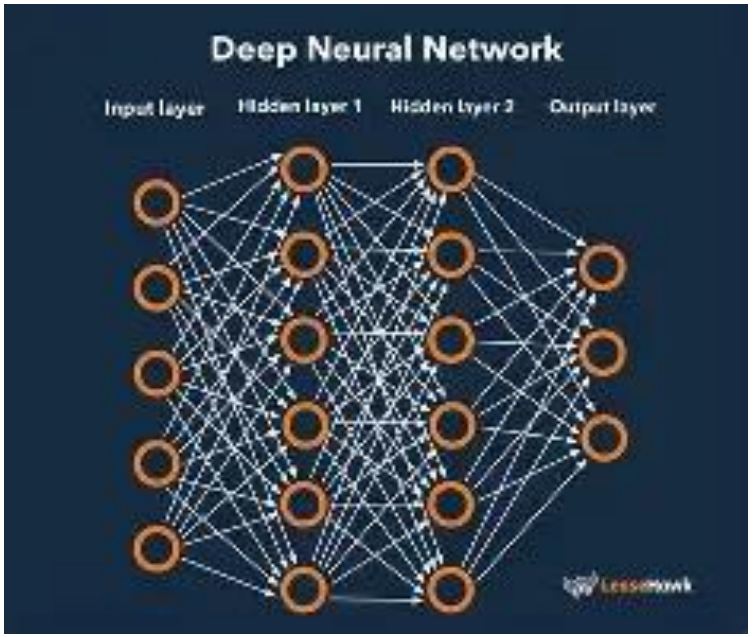

Le Gradient Boosting, c'est aussi un algorithme supervisé pour la régression.

Crée les arbres un par un, et chaque nouvel arbre corrige les erreurs des arbres précédents.

- Le modèle apprend donc progressivement, en se concentrant à chaque fois sur ce qu'il a mal prédit.
- À la fin, toutes les prédictions sont additionnées, avec des poids, pour donner le résultat final.

C'est un modèle très précis, mais plus sensible au surapprentissage si on le règle mal.

# IA de prédiction de température




Le réseau de neurones, c'est un modèle inspiré du cerveau humain. Il est composé de plusieurs couches de neurones qui prennent des données en entrée, les transforment petit à petit, et donnent une prédiction à la fin.

- Chaque neurone fait un petit calcul, et plus on avance dans les couches, plus les infos sont travaillées.
- Le modèle apprend en ajustant les connexions entre neurones pour réduire l'erreur de prédiction.

C'est puissant pour des problèmes complexes, surtout quand les relations entre les données ne sont pas évidentes.

# IA de prédiction de température

## 4. Sur quoi se base le code pour choisir la meilleur IA ?

```
print(f"\n GAGNANT: {self.meilleur_modele_nom}")
print(f" Précision (R²): {self.precision_courante:.3f}")
print(f" Erreur moyenne: {self.erreur_moyenne:.3f}°C")
```

Comparaison avec deux indicateurs : le MAE, moyenne des écarts absolus (le +bas),qui mesure l'erreur moyenne, et le R², coefficient de détermination (+proche de 1), indique la qualité globale de la prédiction. Ensuite, il choisit le modèle qui a le meilleur compromis entre les 2 et sauvegarde avec .joblib pour l'utiliser dans notre script sur la raspberry pi.

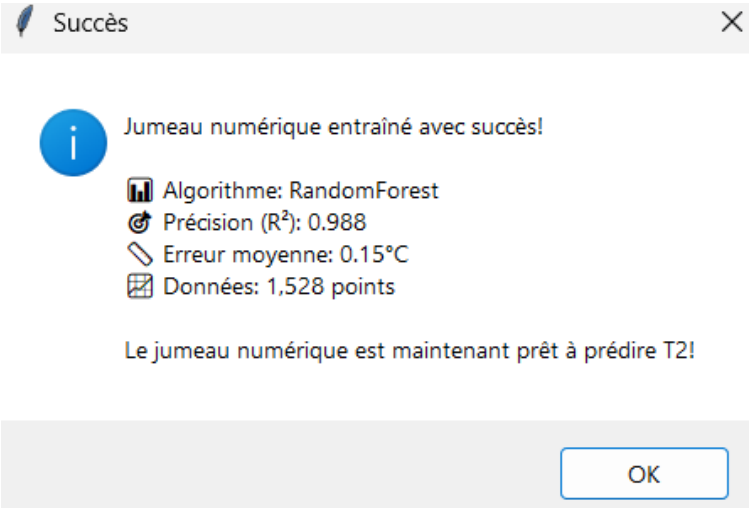
```
# Sauvegarde des modèles
try:
    joblib.dump(self.modele_prediction, 'jumeau_numerique_t2.pkl')
    joblib.dump(self.scaler, 'scaler_jumeau.pkl')
    print("💾 Jumeau numérique sauvegardé")
except Exception as e:
    print(f"⚠️ Erreur sauvegarde: {e}")

return True
```

Test des algorithmes...
📈 RandomForest : MAE=0.252°C, R²=0.867
📈 GradientBoosting: MAE=0.263°C, R²=0.855
📈 NeuralNetwork : MAE=0.432°C, R²=0.059
🏆 GAGNANT: RandomForest
🎯 Précision (R²): 0.867
📏 Erreur moyenne: 0.252°C
💾 Jumeau numérique sauvegardé

# IA de prédiction de température

## 5. Comment fonctionne-l'IA ?




```python
try:
    if not self.modele_actif or self.modele_prediction is None:
        return None

    # 🔧 Préparation des features en temps réel
    features_base = [
        t1, t3,                                # Températures actuelles
        puissance_chauffe, puissance_vent,     # Puissances actuelles
        1 if chauffage_on else 0,              # États binaires
        1 if ventilateur_on else 0
    ]
```

Une fois que le meilleur modèle est choisi, c'est lui qu'on utilise tout le long du projet. À chaque seconde, on lui envoie les données en direct (température, puissance, consigne…). Il les transforme, les analyse et prédit la température. Ça permet de voir à l'avance comment le système va évoluer, et potentiellement de prendre des décisions pour anticiper.

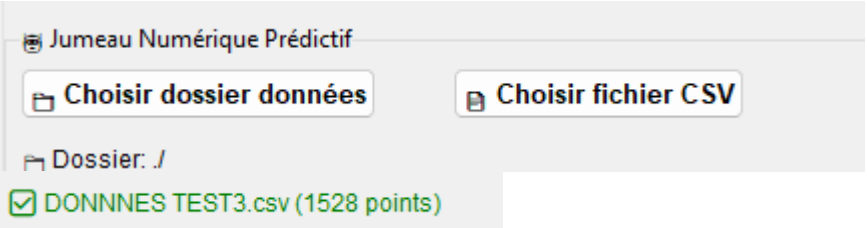
# Importation Fichier CSV IA

- Récupération du fichier csv, analyse et entraînement

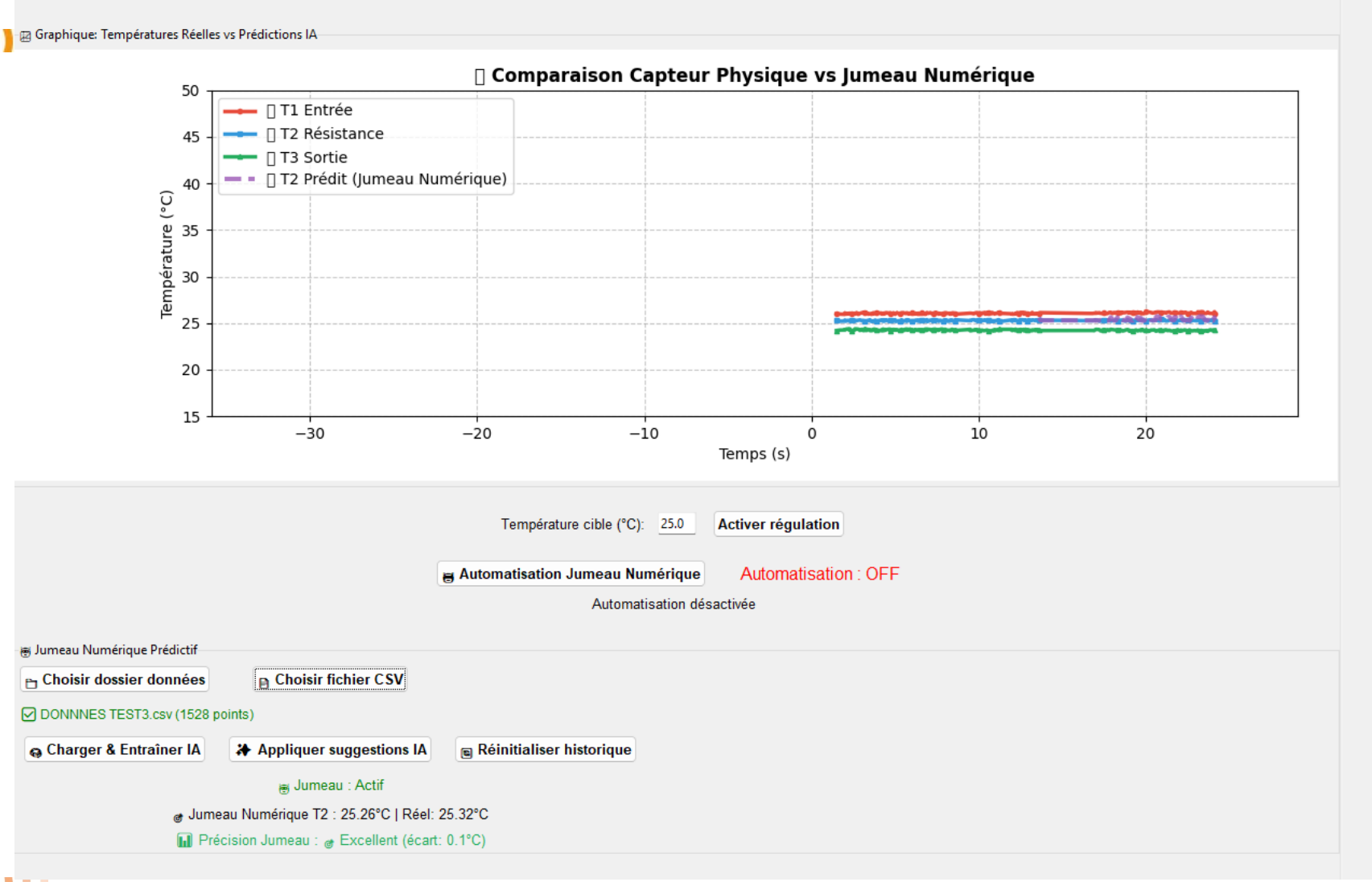
simulations-

| Temps | T1 | T2 | T3 | Consigne | Puissance Ch | Puissance Ve | Ventilateur O | Chauffage ON | Regulation Active |
|---|---|---|---|---|---|---|---|---|---|
| 2.2 | 25.93 | 25.16 | 24.26 | 25.0 | 0.0 | 0.0 | 0 | 0 | 0 |
| 2.4 | 26.02 | 25.33 | 24.33 | 25.0 | 0.0 | 0.0 | 0 | 0 | 0 |
| 2.6 | 25.98 | 25.28 | 24.25 | 25.0 | 0.0 | 0.0 | 0 | 0 | 0 |
| 2.8 | 26.01 | 25.27 | 24.36 | 25.0 | 0.0 | 0.0 | 0 | 0 | 0 |
| 3.0 | 26.11 | 25.31 | 24.19 | 25.0 | 0.0 | 0.0 | 0 | 0 | 0 |
| 3.1 | 25.92 | 25.34 | 24.27 | 25.0 | 0.0 | 0.0 | 0 | 0 | 0 |
| 3.3 | 26.0 | 25.2 | 24.17 | 25.0 | 0.0 | 0.0 | 0 | 0 | 0 |
| 3.4 | 25.94 | 25.21 | 24.26 | 25.0 | 0.0 | 0.0 | 0 | 0 | 0 |
| 3.5 | 25.93 | 25.3 | 24.29 | 25.0 | 0.0 | 0.0 | 0 | 0 | 0 |
| 3.7 | 26.07 | 25.3 | 24.29 | 25.0 | 0.0 | 0.0 | 0 | 0 | 0 |
| 3.9 | 26.0 | 25.31 | 24.22 | 25.0 | 0.0 | 0.0 | 0 | 0 | 0 |
| 4.0 | 25.99 | 25.31 | 24.23 | 25.0 | 0.0 | 0.0 | 0 | 0 | 0 |

- Récupération des fichiers pour faire les prédictions

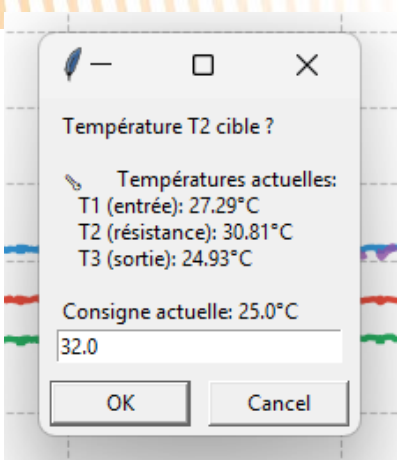


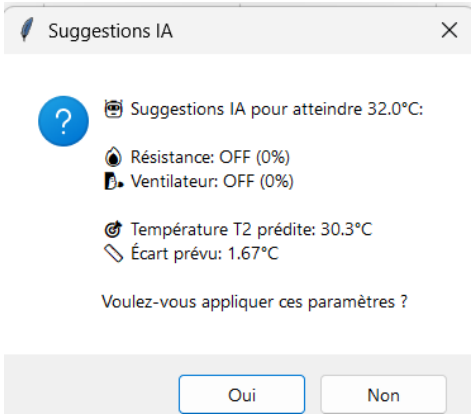
# IA IHM

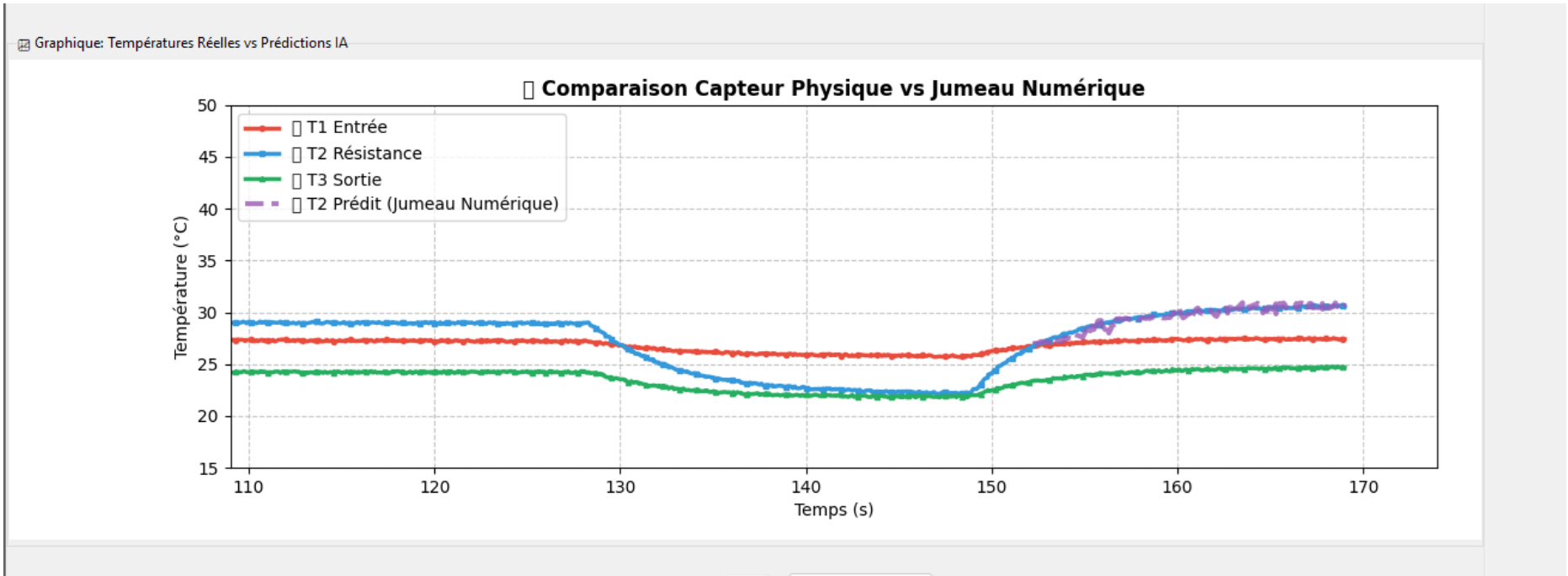

Graphique: Températures Réelles vs Prédictions IA
Comparaison Capteur Physique vs Jumeau Numérique
T1 Entrée
T2 Résistance
T3 Sortie
T2 Prédit (Jumeau Numérique)
Température (°C)
Temps (s)
Température cible (°C): 25.0
Activer régulation
Automatisation Jumeau Numérique
Automatisation : OFF
Automatisation désactivée
Jumeau Numérique Prédictif
Choisir dossier données
Choisir fichier CSV
DONNNES TEST3.csv (1528 points)
Charger & Entraîner IA
Appliquer suggestions IA
Réinitialiser historique
Jumeau : Actif
Jumeau Numérique T2 : 25.26°C | Réel: 25.32°C
Précision Jumeau : Excellent (écart: 0.1°C)

# Analyse Prédictions

## Sous forme de graphe



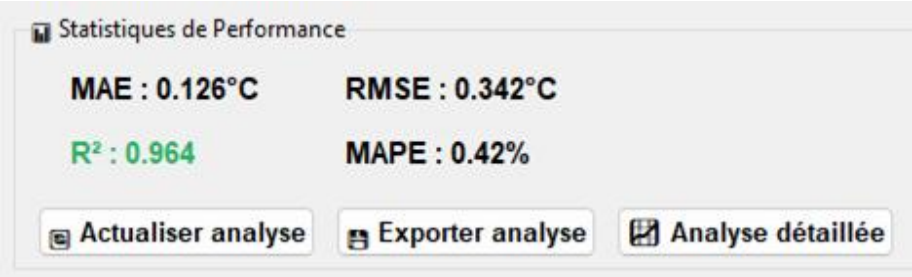



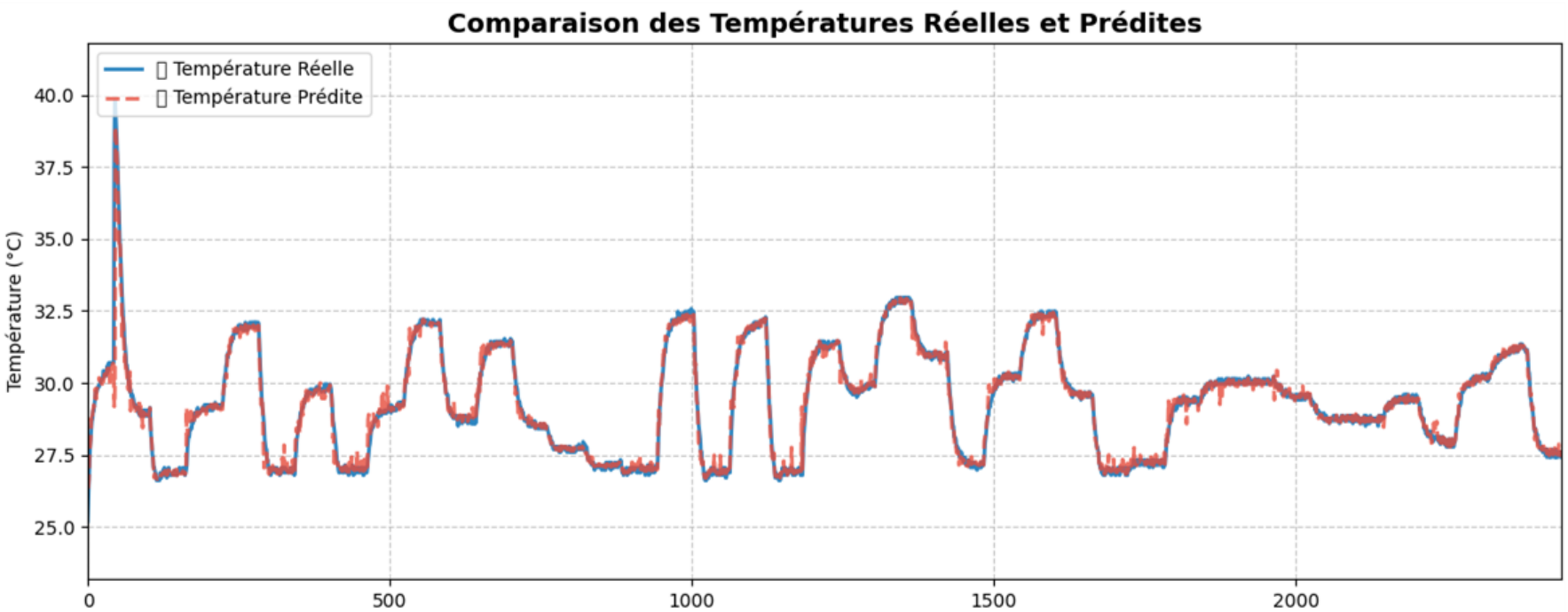



# Analyse Prédictions

## Comparaison des Températures

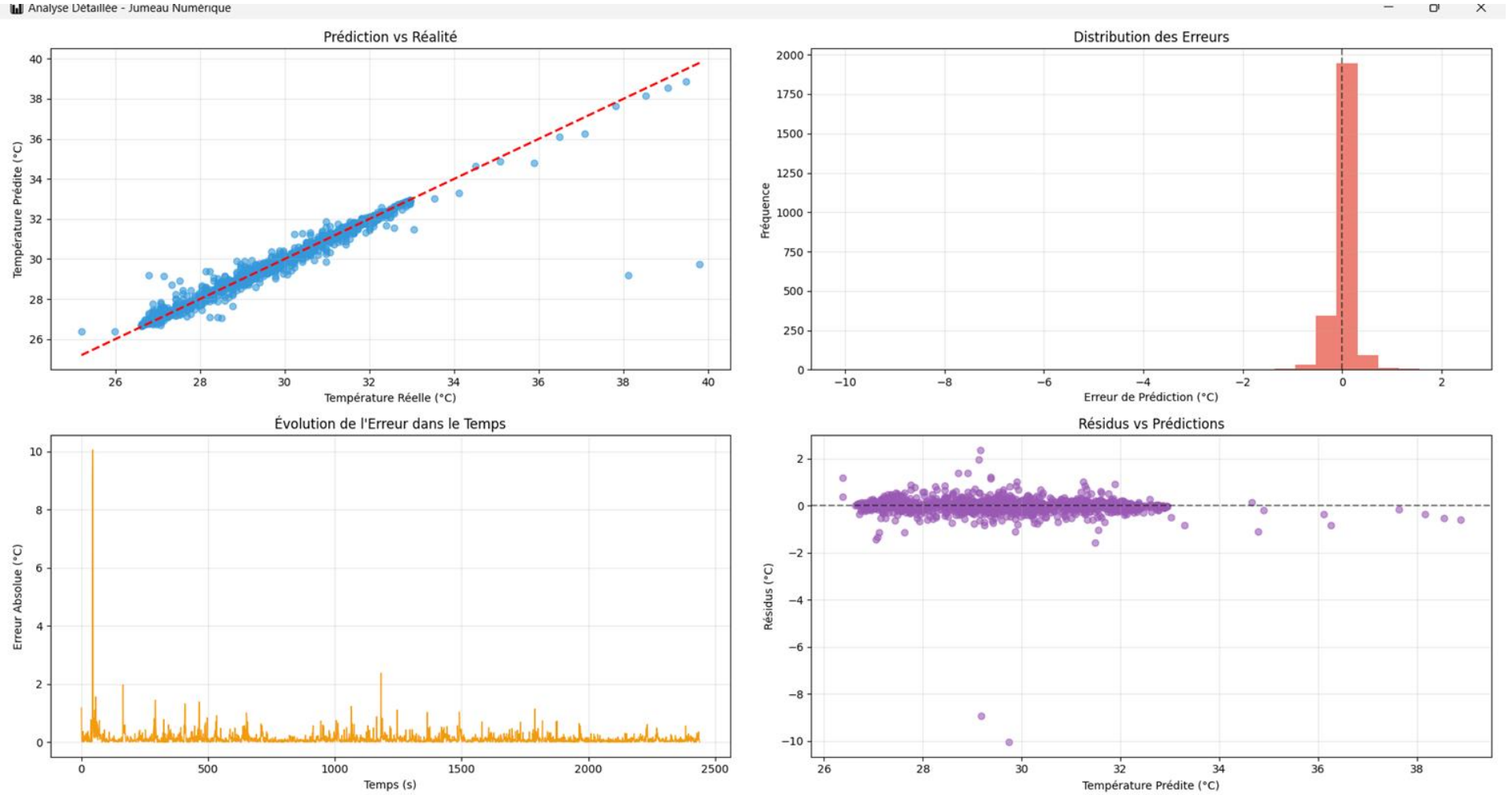
Évalue la qualité de ton modèle IA



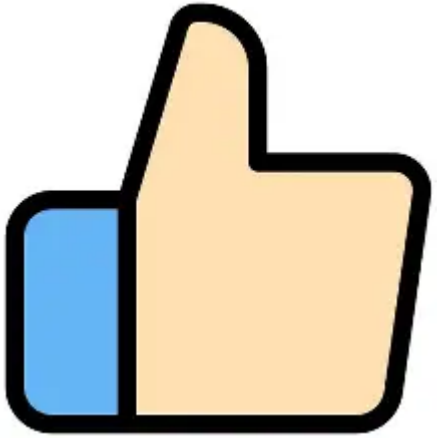
# Autres graphiques

## Sous forme de graphe



# Limites et amélioration IA

1. **Diversité limitée :**
   - 13 combinaisons sur 2440 points
   - Peu d'utilisation du Neuronal Network
   - Biais 89% ventilateur ON
2. **Plages restreintes :**
   - T1: 23-28°C seulement
   - Consigne fixe à 27°C
   - Risque d'extrapolation
3. **Déséquilibre :**
   - 89% ventilateur ON vs 11% OFF
4. **Réduire les erreurs d'entrées:**
   - Nettoyer les données
   - Ajouter plus de données d'apprentissage
   - Enrichir les variables (features)
   - Choisir un meilleur modèle



**Solutions implémentées :**

1. **Augmentation de données** par interpolation
2. **Rééquilibrage** des classes minoritaires
3. **Ensemble de modèles** (RandomForest + GradientBoosting + Neural Network)
4. **Détection d'extrapolation** automatique
5. **Estimation d'incertitude** pour chaque prédiction

# Site WEB

→ Création du site internet sur VSC en HTML
→ Screen vidéo présent sur le drive

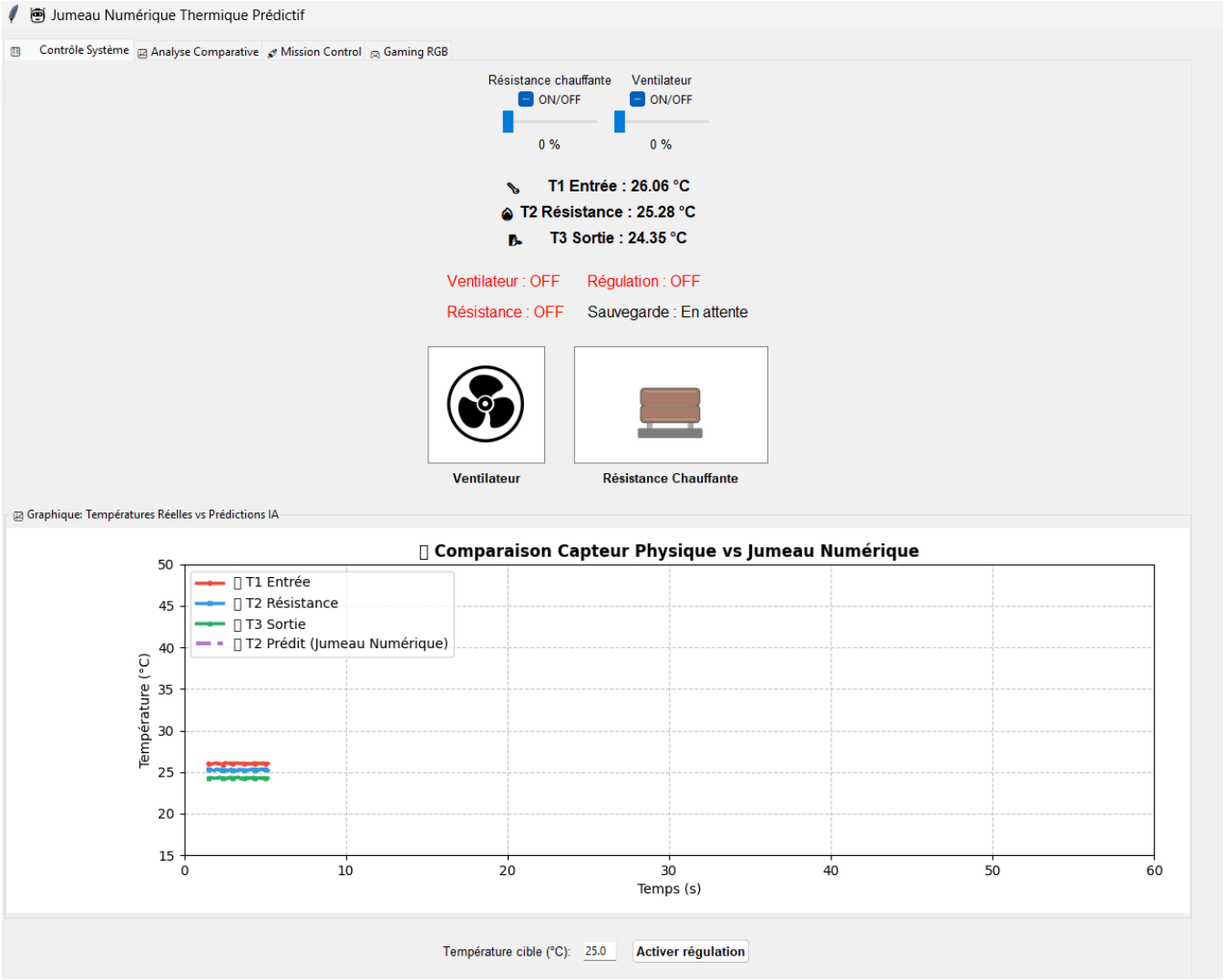
# Conclusion

- ➔ Système physique fonctionnel
- ➔ Jumeau numérique identique
- ➔ Prédiction de la température à l'aide de l'IA
- ➔ Apprentissage constant : nouvelles notions

Ce projet nous a permis d'approfondir notre compréhension des systèmes cyber-physiques tout en développant nos compétences en modélisation, en instrumentation et en travail d'équipe.

# Merci de nous avoir écoutés

# Annexes


Jumeau Numérique Thermique Prédictif
Contrôle Système
Analyse Comparative
Mission Control
Gaming RGB
Résistance chauffante
ON/OFF
0 %
Ventilateur
ON/OFF
0 %
T1 Entrée : 26.06 °C
T2 Résistance : 25.28 °C
T3 Sortie : 24.35 °C
Ventilateur : OFF
Régulation : OFF
Résistance : OFF
Sauvegarde : En attente
Ventilateur
Résistance Chauffante
Graphique: Températures Réelles vs Prédictions IA
Comparaison Capteur Physique vs Jumeau Numérique
T1 Entrée
T2 Résistance
T3 Sortie
T2 Prédit (Jumeau Numérique)
Température (°C)
Temps (s)
Température cible (°C):
25.0
Activer régulation

# d) Programmation

```python
"""
Interface graphique de jumeau numérique thermique prédictif pour Raspberry Pi
--------------------------------------------------------------------------------
🎯 OBJECTIF: Créer un jumeau numérique qui prédit T2 (température résistance/processeur)
             à partir des paramètres mesurés (T1, T3, puissances, états)

🧠 IA PRÉDICTIVE: Utilise l'apprentissage automatique pour prédire T2 en temps réel
                  et suggérer les meilleurs paramètres pour atteindre une consigne

📊 FONCTIONNALITÉS:
- Contrôle PWM résistance chauffante et ventilateur
- Lecture 3 capteurs NTC via MCP3008 (T1, T2, T3)
- Jumeau numérique prédictif avec réseau de neurones
- Graphique temps réel: températures réelles vs prédictions IA
- Optimisation automatique des paramètres
- Sauvegarde continue des données pour apprentissage

🔧 MODIFICATIONS:
- Enregistrement données: toutes les 10s (au lieu de 0.3s)
- Sauvegarde sécurité: toutes les 10 minutes
- Ventilateur automatisation: 66-100% (au lieu de 10-100%)
"""

# =============== IMPORTS NÉCESSAIRES ===============
import tkinter as tk
from tkinter import ttk
import time
import threading
import csv
import datetime
import random
import matplotlib.pyplot as plt
import os
import math
from PIL import Image, ImageTk
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import pandas as pd
```

```python
# =============== IMPORTS NÉCESSAIRES ===============
import tkinter as tk
from tkinter import ttk
import time
import threading
import csv
import datetime
import random
import matplotlib.pyplot as plt
import os
import math
from PIL import Image, ImageTk
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, r2_score
from sklearn.preprocessing import StandardScaler
import joblib
import tkinter.filedialog as filedialog
import tkinter.messagebox as messagebox
import glob
```

```python
# =============== CONFIGURATION RASPBERRY PI ===============
try:
    import RPi.GPIO as GPIO
    import spidev
    RPI = True
    print("🔧 Mode Raspberry Pi détecté")
except ImportError:
    RPI = False
    print("💻 Mode simulation PC")

if RPI:
    # Configuration broches GPIO
    VENT_IN1 = 16    # PWM ventilateur
    VENT_IN2 = 26    # Direction ventilateur (fixe LOW)
    HEAT_IN3 = 5     # PWM résistance chauffante
    HEAT_IN4 = 6     # Direction résistance (fixe LOW)

    GPIO.setmode(GPIO.BCM)
    GPIO.setup(VENT_IN1, GPIO.OUT)
    GPIO.setup(VENT_IN2, GPIO.OUT)
    GPIO.setup(HEAT_IN3, GPIO.OUT)
    GPIO.setup(HEAT_IN4, GPIO.OUT)

    # Initialisation PWM
    pwm_vent = GPIO.PWM(VENT_IN1, 100)
    pwm_chauffe = GPIO.PWM(HEAT_IN3, 100)
    pwm_vent.start(0)
    pwm_chauffe.start(0)

    # Configuration SPI pour MCP3008
    spi = spidev.SpiDev()
    spi.open(0, 0)
    spi.max_speed_hz = 1350000
```

# d) Programmation

```python
# =============== JUMEAU NUMÉRIQUE PRÉDICTIF ===============
class JumeauNumeriquePredict:
    """
    JUMEAU NUMÉRIQUE PRÉDICTIF pour la température T2
    OBJECTIF: Prédire T2 (température résistance/processeur) en fonction de:
    - T1 (température entrée)
    - T3 (température sortie)
    - Puissance chauffage (0-100%)
    - Puissance ventilateur (0-100%)
    - États ON/OFF des équipements
    ALGORITHMES: RandomForest, GradientBoosting, Réseau de neurones
    📊 APPRENTISSAGE: Sur données historiques CSV
    ⚡ PRÉDICTION: Temps réel avec comparaison vs capteur physique
    """
    def __init__(self): ···

    def detecter_format_csv(self, fichier_csv): ···

    def charger_donnees_csv(self, fichier_specifique=None): ···

    def valider_donnees_csv(self, df): ···

    def preparer_features_prediction(self, df): ···

    def entrainer_modele_comportemental(self): ···

    def predire_t2_temps_reel(self, t1, t3, puissance_chauffe, puissance_vent, chauffage_on, ventilateur_on): ···

    def ajouter_mesure_temps_reel(self, temps, t2_reel, t2_predit=None): ···

    def obtenir_donnees_comparaison(self): ···

    def suggerer_parametres_optimaux(self, temperature_cible, t1_actuel, t3_actuel): ···

    def definir_dossier_donnees(self, dossier): ···

    def reinitialiser_historique(self): ···
```

# d) Programmation

```python
# ============== INTERFACE GRAPHIQUE PRINCIPALE ======
class ControleThermique:
    """...

    def __init__(self, root): ...

    def setup_scrollable_interface(self): ...

    def create_widgets(self): ...

    def creer_graphique_controle(self): ...

    def creer_animations(self): ...

    def setup_fan_animation(self): ...

    def create_default_fan(self): ...

    def draw_fan_blades(self): ...

    def setup_heater_animation(self): ...

    def create_default_heater(self): ...

    def rotate_fan(self): ...

    def update_heater_animation(self): ...

    def update_heater_glow(self, opacity): ...

    def create_analyse_tab(self): ...

    def creer_interface_nasa_style(self): ...

    def creer_panel_telemetrie(self, parent): ...
```

```python
    def rotate_fan(self): ...

    def update_heater_animation(self): ...

    def update_heater_glow(self, opacity): ...

    def create_analyse_tab(self): ...

    def creer_interface_nasa_style(self): ...

    def creer_panel_telemetrie(self, parent): ...

    def creer_radar_thermique(self, parent): ...

    def creer_panel_commandes_nasa(self, parent): ...

    def emergency_stop(self): ...

    def dessiner_grille_radar(self): ...

    def creer_interface_gaming(self): ...

    def creer_panel_stats_gaming(self, parent): ...

    def creer_barre_stat_gaming(self, parent, name, min_val, max_val, color): ...

    def activer_toutes_interfaces(self): ...

    def creer_graphique_analyse(self): ...
```

# d) Programmation

```python
# =============== MÉTHODES JUMEAU NUMÉRIQUE ===============

def choisir_fichier_csv(self): ...

def choisir_dossier_donnees(self): ...

def charger_et_entrainer_jumeau(self): ...

def appliquer_suggestions_ia(self): ...

def appliquer_parametres_ia(self, suggestions): ...

def generer_suggestions_manuelles(self): ...

def reinitialiser_jumeau(self): ...

def charger_config_ia(self, fichier_config="config_ia.txt"): ...

def sauvegarder_config_ia(self, fichier_config="config_ia.txt"): ...

# =============== MÉTHODES ANALYSE ===============

def charger_fichier_analyse(self): ...

def generer_predictions_analyse(self): ...

def calculer_metriques_performance(self, donnees=None): ...

def actualiser_analyse(self): ...

def filtrer_donnees_periode(self): ...

def mettre_a_jour_graphiques_analyse(self, donnees): ...
```

```python
def changer_periode_analyse(self, event=None): ...

def analyse_detaillee(self): ...

def exporter_analyse(self): ...

# =============== MÉTHODES ACQUISITION ET CONTRÔLE ===============

def demarrer_acquisition(self): ...

def lire_capteurs(self): ...

def update_predictions_jumeau_numerique(self, t1, t2, t3, temps_actuel): ...

def update_graphique_avec_jumeau(self): ...

def update_loop(self): ...

def mettre_a_jour_interfaces_modernes(self, t1, t2, t3, temps): ...

def mettre_a_jour_radar(self, t1, t2, t3): ...

def mettre_a_jour_stats_gaming(self, t1, t2, t3): ...

# =============== MÉTHODES INTERFACE ===============

def update_pwm_chauffe(self, *args): ...

def update_pwm_vent(self, *args): ...

def toggle_chauffage(self): ...

def toggle_ventilateur(self): ...

def toggle_regulation(self): ...
```

# d) Programmation

```
    def toggle_ventilateur(self): ...

    def toggle_regulation(self): ...

    def toggle_automatisation(self): ...

    # =============== MÉTHODES AUTOMATISATION ===============

    def executer_automatisation(self): ...

    # =============== MÉTHODES SAUVEGARDE ===============

    def creer_fichier_session(self): ...

    def sauvegarde_incrementale(self): ...

    def sauvegarder(self): ...

    def stop(self): ...


# =============== POINT D'ENTRÉE PRINCIPAL ===============

if __name__ == "__main__": ...
```